

SUGGESTED PROJECT LIST

FOR DISTRIBUTED COMPUTING SYSTEMS

September 2026

1. Suggested Topic Groups

Topic Group	Projects
Cloud, Containers & Resource Management	1, 2, 3, 4, 5, 11, 12, 26, 29
Edge, IoT & Geo-distributed Systems	6, 28
Reliability, Observability & Security	7, 9, 10, 22, 23, 27
Distributed Data & Storage	8, 13, 16, 20, 21
Distributed Algorithms & Coordination	14, 19, 24, 26, 30
Distributed Processing & Messaging	15, 17, 18
Peer-to-Peer / Decentralized Systems	25

Recommended team size: 3–4 students. Each project should include implementation, repeatable experiments, measurable metrics, analysis, and a final report/demo.

2. Project Summary

#	Project	Difficulty	Main Concepts
1	Cloud-native microservices with autoscaling	★★★	Microservices, containerization, autoscaling, observability
2	Serverless event-driven processing pipeline	★★★	Serverless computing, event-driven architecture, cold start
3	Dynamic Resource Allocation in Cloud Computing / Container-based Platforms	★★★	Resource management, monitoring, adaptive control
4	Load Balancing Techniques for Cloud Resource Management	★★★	Load balancing, scalability, fault tolerance
5	Cost & performance comparison: VM vs container vs serverless	★★★	Cloud execution models, performance benchmarking, cost-performance trade-off
6	Edge IoT pipeline with cloud offloading	★★★	Edge computing, cloud offloading, latency-bandwidth trade-off
7	Hybrid backup and automated failover demo	★★★	Backup, disaster recovery, RTO/RPO, failover
8	Storage benchmarking: object vs block vs file	★★★	Distributed storage, I/O benchmarking, workload-aware storage selection
9	Observability & SRE playbook for a cloud service	★★★	Observability, SLI/SLO, incident response, resilience
10	Secure cloud application: IAM & zero-trust demo	★★★	IAM, least privilege, zero trust, network security
11	Service placement & cost optimiser (sized)	★★★★	Service placement, optimization, latency SLO, cost constraints
12	Autoscaling in Container / Cloud / Edge Simulation Environments	★★★★	Scheduling, placement, autoscaling, simulation
13	Distributed Key-Value Store	★★★	Partitioning, consistent hashing, replication, fault tolerance
14	Leader Election & Distributed Consensus	★★★★	Leader election, consensus, failure recovery, distributed state
15	Mini MapReduce Framework	★★★	Distributed computation, task scheduling, shuffle, fault recovery
16	Distributed Cache & Cache Consistency	★★★	Caching, consistency, invalidation, cache stampede

#	Project	Difficulty	Main Concepts
17	Message Queue & Distributed Task Processing	★★★	Asynchronous messaging, acknowledgement, retry, delivery semantics
18	Real-Time Stream Processing System	★★★★	Event streaming, consumer groups, stream processing, backpressure
19	Distributed Rate Limiter	★★★	Distributed coordination, atomic operations, fairness
20	Distributed File Storage System	★★★★	Chunking, replication, metadata, recovery
21	Consistency Models in Distributed Databases	★★★★	Strong consistency, eventual consistency, CAP, stale reads
22	Fault Injection & Chaos Engineering	★★★★	Fault tolerance, recovery, resilience testing, chaos engineering
23	Circuit Breaker & Resilient Microservices	★★★	Timeouts, retries, circuit breaker, cascading failure
24	Service Discovery in Microservice Systems	★★★	Service registry, heartbeat, health check, dynamic discovery
25	Peer-to-Peer File Sharing System	★★★★	P2P, decentralized discovery, chunk exchange, parallel download
26	Distributed Job Scheduler	★★★	Scheduling, resource allocation, fairness, bin packing
27	Distributed Log Aggregation System	★★★	Log streaming, buffering, indexing, distributed ingestion
28	Geo-Distributed CDN Simulator	★★★	Content delivery, geo-routing, caching, replication
29	Multi-Cloud Deployment & Failover	★★★★	Multi-cloud, high availability, traffic failover, disaster recovery
30	CRDT-based Collaborative Application	★★★★★	CRDT, eventual consistency, conflict-free replication, offline collaboration

3. Detailed Project Descriptions

1 — Cloud-native microservices with autoscaling

Suggested tasks / requirements

- Design a 3-service microservice app (API, catalogue, DB).
- Containerise each service and provide a Helm chart or Kubernetes manifests.
- Deploy to a Kubernetes cluster and implement HPA (Horizontal Pod Autoscaler).
- Add a simple Prometheus metric and a Grafana dashboard; run a load experiment and record autoscaling behaviour.
- Produce a final report with measurements and lessons learned.

Tools: Docker, Kubernetes (Minikube/GKE), Prometheus, Grafana.

2 — Serverless event-driven processing pipeline

Suggested tasks / requirements

- Design an event flow: uploader → processor → store.
- Implement minimal functions (upload + transform) and containerise local versions.
- Deploy serverless functions (or OpenFaaS) and set up event triggers.
- Measure latency and cost proxies such as invocation time and cold starts.
- Report results and recommendations.

Tools: AWS Lambda / Cloud Functions or OpenFaaS, S3/MinIO.

3 — Dynamic Resource Allocation in Cloud Computing / Container-based Platforms

Suggested tasks / requirements

- Monitor CPU and memory usage across distributed nodes.
- Define load thresholds to trigger resource reallocation.
- Implement a controller that reallocates resources among running containers.
- Dynamically adjust container resource limits at runtime.
- Compare static vs dynamic resource allocation.

Tools: Docker, Kubernetes, Minikube, Prometheus, Python.

4 — Load Balancing Techniques for Cloud Resource Management

Suggested tasks / requirements

- Implement and compare multiple load-balancing algorithms in a distributed environment.
- Compare algorithms such as Round Robin, Least Connections, and Weighted Load Balancing.
- Discuss distributed load distribution, fault tolerance, and scalability.
- Report and evaluate throughput, response time, and load-distribution fairness.

Tools: NGINX / HAProxy, Docker Compose, Kubernetes, Python-based backend services.

5 — Cost & performance comparison: VM vs container vs serverless

Suggested tasks / requirements

- Define a fixed benchmark workload (web or CPU/I/O job).
- Implement the same workload as a VM service, container service, and serverless function.
- Run standardised experiments under identical load profiles.
- Produce cost/latency/throughput comparisons and charts.
- Draw conclusions and recommendations.

Tools: EC2/GCE or local VMs, Docker, Lambda/Cloud Run or OpenFaaS, wrk/sysbench.

6 — Edge IoT pipeline with cloud offloading

Suggested tasks / requirements

- Simulate sensor producers and a lightweight edge pre-processor.
- Implement an offload policy (threshold-based or fixed fraction).
- Deploy an edge component (container) and cloud backend.
- Measure bandwidth saved, end-to-end latency, and accuracy if applicable.
- Report trade-offs and policy tuning.

Tools: MQTT, Python/Node edge service, MinIO or cloud storage.

7 — Hybrid backup and automated failover demo

Suggested tasks / requirements

- Design a hybrid backup architecture (local VM + cloud).
- Implement automated backup (snapshot/script) and cloud restore procedure.
- Automate a failover trigger.
- Measure RTO/RPO under a scheduled failover test.
- Produce a playbook and final report.

Tools: VirtualBox/VM, S3/MinIO, Terraform/Ansible optional.

8 — Storage benchmarking: object vs block vs file

Suggested tasks / requirements

- Pick three representative workloads such as static CDN, database, and random I/O.
- Implement simple drivers for each workload targeting object, block, and file stores.
- Run standard benchmarks for latency, throughput, and cost proxy.
- Analyse results and create a workload-to-storage recommendation table.
- Write up results and caveats.

Tools: MinIO (S3), local block volumes/persistent volumes, NFS/EFS (or simulated).

9 — Observability & SRE playbook for a cloud service

Suggested tasks / requirements

- Deploy a small service and instrument it with metrics, logs, and traces.
- Define two SLIs, one SLO, and two alerting rules.
- Run a controlled fault injection and record observability outputs.
- Prepare an on-call runbook and incident postmortem template.
- Submit dashboards, playbook, and report.

Tools: Prometheus, Grafana, Loki/ELK, Jaeger, simple scripted chaos test.

10 — Secure cloud application: IAM & zero-trust demo

Suggested tasks / requirements

- Implement a small multi-role app (for example, admin/user APIs).
- Design IAM roles and least-privilege policies.
- Apply network policies and demonstrate encryption at rest.
- Perform a basic threat model and mitigation checklist; run a security scan.
- Deliver a report with proof of concept.

Tools: Kubernetes RBAC/IAM, KMS/Vault (demo), simple static scanner.

11 — Service placement & cost optimiser (sized)

Suggested tasks / requirements

- Define a simulated demand map across three regions.
- Formulate a small optimisation problem (ILP or greedy heuristic) to place 1–3 services subject to latency SLO and cost budget.
- Implement the optimiser and a simple web UI showing placements.
- Run sensitivity analysis for demand/cost changes.
- Write results and recommended policies.

Tools: Python, PuLP/CVXPY (or simple heuristic), Flask for UI.

12 — Autoscaling in Container / Cloud / Edge Simulation Environments

Suggested tasks / requirements

- Design a simplified scheduler model with resources, constraints, and preferences.
- Implement a scheduler simulator with a placement algorithm such as bin-pack or round-robin with fairness.
- Decide where tasks should be processed in a Cloud/Edge cluster.
- Scale experiments in Cloud/Edge simulation environments to test distributed-system workloads.
- Test on synthetic task traces and report placement efficiency and fairness metrics.
- Compare with a simple baseline such as random or naive placement.
- Present findings and complexity analysis.

Tools: CloudSim, CloudSimPlus, iFogSim, EdgeCloudSim, Java, Python.

13 — Distributed Key-Value Store

Suggested tasks / requirements

- Build a key-value service distributed across 3–5 nodes.
- Implement consistent hashing or another partitioning strategy.
- Implement configurable replication across nodes.
- Support node join/leave and basic failure detection.
- Run experiments with node shutdown/recovery.
- Evaluate throughput, latency, availability, and data-distribution balance.

Tools: Python or Go, Docker, REST/gRPC.

14 — Leader Election & Distributed Consensus

Suggested tasks / requirements

- Build a cluster of 3–5 processes or containers.
- Implement a leader-election algorithm such as Bully or Ring.
- Advanced option: implement a simplified Raft leader-election/log-replication component.
- Simulate leader crash, network delay, and node recovery.
- Measure election/recovery time and behaviour under failures.
- Compare at least two election strategies or configurations.

Tools: Python or Go, Docker, gRPC.

15 — Mini MapReduce Framework

Suggested tasks / requirements

- Implement a master that distributes map and reduce tasks to workers.
- Implement Map, Shuffle/Intermediate Results, and Reduce stages.
- Support worker failure and task reassignment.
- Test using workloads such as Word Count, log analysis, URL frequency, or inverted index.
- Measure speedup, task completion time, and fault-recovery overhead.

Tools: Python or Go, Docker, sockets/gRPC.

16 — Distributed Cache & Cache Consistency

Suggested tasks / requirements

- Deploy multiple API instances sharing a distributed cache.
- Implement or compare Cache-Aside, Write-Through, Write-Back, and TTL strategies.
- Create scenarios producing stale cache and cache invalidation.
- Investigate cache stampede and one mitigation mechanism.
- Measure latency, database workload, throughput, and cache hit ratio.

Tools: Redis, Docker, Python/Node.js, Prometheus.

17 — Message Queue & Distributed Task Processing

Suggested tasks / requirements

- Build producers, a message broker, and multiple workers.
- Implement acknowledgement and retry handling.
- Configure or implement a dead-letter queue.
- Experiment with worker failure and duplicate messages.
- Use a realistic job such as image processing, email processing, document conversion, or video transcoding.
- Measure queue delay, throughput, failure recovery, and duplicate-processing behaviour.

Tools: RabbitMQ, Kafka or Redis Streams, Docker.

18 — Real-Time Stream Processing System

Suggested tasks / requirements

- Generate a continuous stream such as IoT sensor, network, server metric, or vehicle data.
- Publish events through Kafka or an equivalent broker.
- Process the stream using windowing or aggregation.
- Store results and visualize them on a dashboard.
- Experiment with increasing event rates and consumer slowdowns.
- Measure event throughput, end-to-end latency, and consumer lag.

Tools: Kafka, Apache Flink or Spark Streaming, Grafana.

19 — Distributed Rate Limiter

Suggested tasks / requirements

- Implement Fixed Window, Sliding Window, Token Bucket, and/or Leaky Bucket algorithms.
- Deploy multiple API server instances behind a load balancer.
- Use a shared distributed state store to enforce global limits.
- Test burst traffic, concurrent clients, and server failures.
- Compare accuracy, throughput, scalability, and fairness.

Tools: Redis, Python/Go, Docker, Locust/k6.

20 — Distributed File Storage System

Suggested tasks / requirements

- Implement upload, file chunking, and distribution across multiple storage nodes.
- Maintain metadata mapping file chunks to nodes.
- Implement replication and download/reconstruction.
- Simulate storage-node failure and recover files from replicas.
- Measure storage overhead, upload/download throughput, and recovery time.

Tools: Python or Go, Docker, REST/gRPC.

21 — Consistency Models in Distributed Databases

Suggested tasks / requirements

- Deploy or implement a replicated data service on multiple nodes.
- Create experiments for strong consistency and eventual consistency.
- Introduce network delay or temporary partition between replicas.
- Measure stale reads, write/read latency, and availability.
- Explain observed trade-offs using consistency and CAP concepts.

Tools: PostgreSQL replicas, Cassandra, Redis, or a custom Python implementation.

22 — Fault Injection & Chaos Engineering

Suggested tasks / requirements

- Deploy a small microservice application with at least three components.
- Inject pod/process crashes, CPU overload, network delay, packet loss, and dependency failure.
- Record request failure rate, availability, latency, and recovery time.
- Introduce mitigation mechanisms such as retry, timeout, or redundancy.
- Compare system behaviour before and after resilience improvements.

Tools: Kubernetes, Chaos Mesh/LitmusChaos or scripted faults, Prometheus, Grafana.

23 — Circuit Breaker & Resilient Microservices

Suggested tasks / requirements

- Build a chain of two or more dependent microservices.
- Create scenarios where a downstream service becomes slow or unavailable.
- Implement timeout, retry with exponential backoff, circuit breaker, and fallback.
- Compare the system with and without resilience patterns.
- Measure latency, error rate, recovery time, and cascading-failure behaviour.

Tools: Spring Boot/Resilience4j, Python or Go, Docker.

24 — Service Discovery in Microservice Systems

Suggested tasks / requirements

- Deploy multiple dynamic service instances.
- Implement service registration and periodic heartbeat.
- Implement health checks and failed-instance removal.
- Implement client-side or server-side discovery.
- Compare discovery latency and behaviour under node churn.
- Optional: compare a custom implementation with Consul or etcd.

Tools: Consul/etcd or custom implementation, Docker, Python/Go.

25 — Peer-to-Peer File Sharing System

Suggested tasks / requirements

- Implement peer discovery or a simple tracker.
- Split files into chunks and exchange chunks between peers.
- Download chunks in parallel and reconstruct the original file.
- Verify chunk/file integrity.
- Test performance as the number of peers increases.
- Analyse download time, network utilization, and peer-failure behaviour.

Tools: Python or Go, sockets/gRPC, Docker optional.

26 — Distributed Job Scheduler

Suggested tasks / requirements

- Model worker nodes with CPU and memory capacities.
- Create jobs with resource demands and execution times.
- Implement algorithms such as First Fit, Best Fit, Round Robin, Least Loaded, or Bin Packing.
- Run the scheduler against real worker containers or a lightweight simulator.
- Measure resource utilization, scheduling latency, rejected jobs, and load fairness.
- Compare at least three scheduling algorithms.

Tools: Python or Go, Docker optional, Prometheus optional.

27 — Distributed Log Aggregation System

Suggested tasks / requirements

- Collect logs from multiple simulated servers/services.
- Stream or buffer logs through a central/distributed ingestion layer.
- Store and index logs for search.
- Build a dashboard or search interface.
- Create a collector or storage failure scenario.
- Measure ingestion throughput, search latency, storage consumption, and loss/recovery behaviour.

Tools: Fluent Bit, Kafka, Elasticsearch/OpenSearch, Grafana.

28 — Geo-Distributed CDN Simulator

Suggested tasks / requirements

- Model clients and edge/cache nodes in multiple geographic regions.
- Implement nearest-node or latency-aware routing.
- Implement cache replacement policies such as LRU, LFU, and FIFO.
- Model origin fetch and cache replication.
- Measure user latency, cache hit ratio, bandwidth consumption, and origin load.
- Compare policies under different traffic distributions.

Tools: Python, SimPy, Docker-based emulation optional.

29 — Multi-Cloud Deployment & Failover

Suggested tasks / requirements

- Deploy the same application to two independent environments or clusters.
- Implement health monitoring and traffic switching/failover.
- Simulate complete failure of one environment.
- Measure failover time, downtime, latency, and a simple cost proxy.
- Document operational complexity and trade-offs of multi-cloud deployment.

Tools: Kubernetes, Terraform optional, Prometheus, DNS/load-balancer solution.

30 — CRDT-based Collaborative Application

Suggested tasks / requirements

- Build a small collaborative application with replicated state.
- Implement a basic CRDT such as G-Counter, PN-Counter, or LWW Register.
- Allow multiple clients to update state concurrently.
- Simulate disconnection, concurrent updates, and reconnection.
- Verify convergence without centralized conflict resolution.
- Advanced option: extend to collaborative text or a richer shared data structure.

Tools: JavaScript/TypeScript or Python, WebSocket, optional CRDT library for comparison.

4. Assessment Rubric

Criterion	Weight
System design & distributed-systems concepts	20%
Implementation quality & reproducibility	25%
Experimental design & metrics	20%
Analysis, discussion & comparison with baseline	20%
Demo, report & presentation	15%

5. General Deliverables

- Source code and deployment/run instructions.
- Architecture diagram and explanation of distributed components.

- At least one baseline or alternative approach for comparison.
- Repeatable experiment scripts and workload description.
- Metrics, tables/charts, and interpretation of results.
- Failure or stress scenario where applicable.
- Final technical report and live/demo video or presentation.

Note: Project scope should be adjusted according to team size, course duration, and students' prior experience. For advanced teams, require stronger failure models, larger experiments, or a comparison against an established system/library.